

Technical Engineering Report: Fine-Tuning Mistral-7B via QLoRA

Project: Assignment 6 - Fine-tuning Mistral with LoRA using HuggingFace + PEFT Date: May 16, 2026 Environment: Google Colab (T4 / A100 GPU) & Streamlit Community Cloud

1. Executive Summary

This report outlines the methodology, architecture, and deployment strategy for fine-tuning the Mistral-7B-Instruct Large Language Model (LLM). Given the hardware constraints of consumer-grade GPUs, the project employs Quantized Low-Rank Adaptation (QLoRA). This approach reduces the memory footprint of the 7-billion parameter model by over 70%, enabling full instruction-tuning on a free Google Colab instance, while maintaining near 16-bit performance fidelity.

2. Problem Statement

Fine-tuning Llama / Mistral with LoRA using HuggingFace + PEFT on Google Colab

The objective was to fine-tune a foundation model (Llama/Mistral) using Low-Rank Adaptation (LoRA) and the HuggingFace Parameter-Efficient Fine-Tuning (PEFT) library. Key requirements included:

- Executing the pipeline within the memory limits of a Google Colab T4 GPU (~15GB VRAM).
- Utilizing bitsandbytes for 4-bit precision loading.
- Training on an instruction-following dataset (OpenAssistant Guanaco).
- Creating a robust, accessible method to demonstrate the fine-tuned model's capabilities.

3. System Architecture & Methodology

3.1 4-bit Quantization (bitsandbytes)

Loading a 7B parameter model in standard FP32 precision requires approximately 28GB of VRAM—far exceeding free-tier hardware capabilities.

- **NF4 Data Type:** The model is loaded using the Normal Float 4-bit (nf4) quantization datatype.
- **Compute Type:** While the weights are stored in 4-bit, the forward/backward passes are computed in float16 to maintain gradient stability and inference speed.

3.2 Low-Rank Adaptation (PEFT / LoRA)

Instead of updating all 7 billion parameters, LoRA freezes the pre-trained model weights and injects trainable rank decomposition matrices into the Transformer architecture.

- **Target Modules:** q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj. Targeting all linear layers maximizes the model's ability to adapt to new linguistic patterns.

- LoRA Hyperparameters:
 - Rank (r) = 16: A balanced rank that captures enough context without bloating the adapter size.
 - Alpha = 32: Scaling factor for the LoRA activations.
 - Dropout = 0.05: Regularization to prevent overfitting on the instruction set.

3.3 Supervised Fine-Tuning (SFTTrainer)

Training was orchestrated using the trl (Transformer Reinforcement Learning) library's SFTTrainer.

- Dataset: timdettmers/openassistant-guanaco (a high-quality, multilingual instruction-following corpus).
- Optimization: paged_adamw_32bit optimizer was utilized to manage memory paging between the GPU and CPU dynamically, preventing Out-Of-Memory (OOM) spikes during backpropagation.

4. Deployment Strategy: The Inference Playground

Because the resulting model cannot be hosted locally on a 1GB Streamlit Cloud server, the project architecture was split into two distinct environments:

1. Training Node (Google Colab): Executes finetune_mistral_lora.py to generate the highly compressed LoRA adapter weights (.safetensors).
2. Inference Interface (Streamlit Cloud): A premium, glassmorphism-styled web application (streamlit_app.py) serves as the "Playground". It utilizes the huggingface_hub SDK to ping the model weights via API, entirely abstracting the hardware requirements away from the end-user.
 - Security: Integrated an ephemeral API key injection field, enforcing zero-trust credential handling.

5. Conclusion & Future Roadmap

Assignment 6 successfully demonstrates an advanced pipeline for customizing state-of-the-art open-source foundation models. The implementation of QLoRA makes local LLM alignment democratic and financially viable.

Future Scalability:

- Custom Datasets: Swap the Guanaco dataset for a proprietary CSV/JSON dataset to train the model on domain-specific knowledge (e.g., medical diagnostics or legal contract parsing).
- DPO (Direct Preference Optimization): Following the SFT phase, apply DPO to align the model's outputs with human preference, significantly reducing hallucinations.

References

1. <https://github.com/jenaarmaan/Intern-Reasearch-Work-Ada-lovelace/tree/assignment-6-lora-finetuning>
2. <https://lora-finetuning.streamlit.app/>